Daeyang University

FACULTY: **FACULTY OF ICT**

PROGRAMME: **PROGRAM**

YEAR: **2**

LECTURER: **Prof. Sanghoon Lee**

FROM: **2420004**

COURSE: **OOP**

COURSE CODE: **BSCICT 2203**

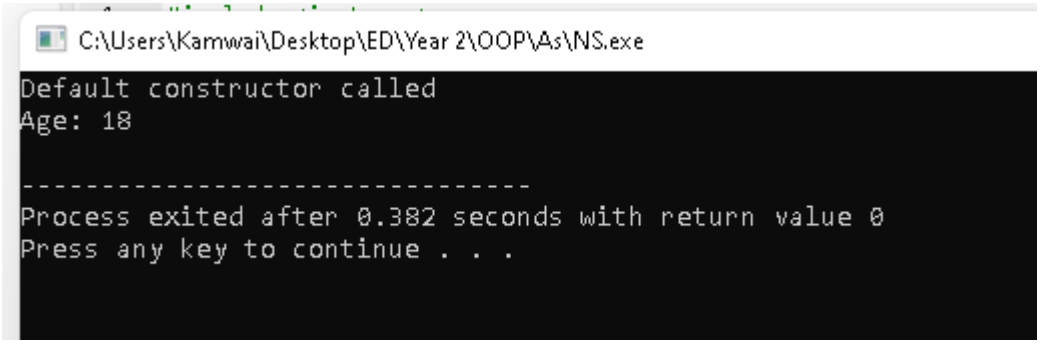
Code

NS.cpp

```cpp
#include <iostream>
using namespace std;

class Student
{
private:
    int age;

public:
    // Default constructor
    Student()
    {
        age = 18;
        cout << "Default constructor called" << endl;
    }

    void display()
    {
        cout << "Age: " << age << endl;
    }
};

int main()
{
    Student s1; // Automatically calls default constructor
    s1.display();

    return 0;
}
```

The program demonstrates the implementation of a default constructor in C++. A class named Student is created with a private variable called age. Inside the public section, a default constructor Student() is defined, which automatically assigns the value 18 to age whenever an object of the class is created. The constructor also displays a message showing that it has been called. A display() function is included to print the value of age. In the main() function, an object s1 is created, which automatically triggers the default constructor, and then the display() function is called to show the stored age value.

OUTPUT



The output first displays the message "Default constructor called" because the constructor executes automatically when the object s1 is created. After that, the program prints "Age: 18" from the display() function, confirming that the constructor successfully initialized the age variable with the default value.

```cpp
#include <iostream>
using namespace std;

// Structure Definition
struct Time {
    int hour;   // 0-23
    int minute; // 0-59
    int second; // 0-59
};

// Function to print time
void printTime(Time t) {
    cout << t.hour << ":" << t.minute << ":" << t.second << endl;
}

int main() {
    // Declaration of variable
    Time lunchTime;

    // Declaration & initialization of variable
    Time dinnerTime = {18, 30, 0};

    // Pointer variable
    Time *ptrTime = &lunchTime;

    // Setting values for lunchTime using direct access
    lunchTime.hour = 12;
    lunchTime.minute = 30;
    lunchTime.second = 20;

    cout << "Lunch will be held at: ";
    printTime(lunchTime);

    cout << "Dinner will be held at: ";
    printTime(dinnerTime);

    return 0;
}
```

This program defines a Time structure to store hours, minutes, and seconds. It then creates two time objects: lunchTime and dinnerTime. A pointer ptrTime is used to access and modify lunchTime indirectly. The function printTime() is used to display both times in a readable format (hour:minute:second).

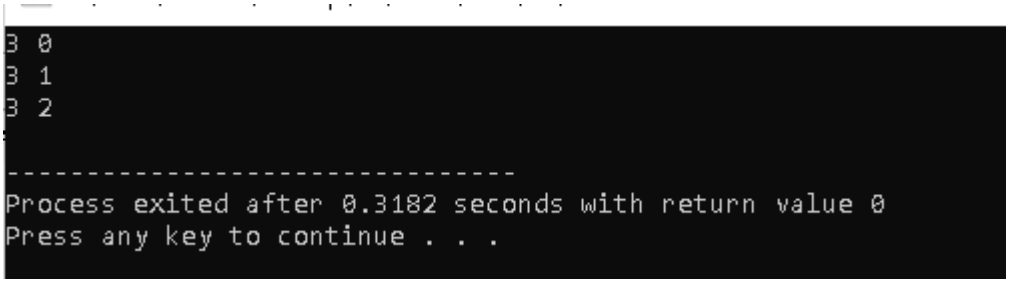
Output:

```
Lunch will be held at: 12:30:20
Dinner will be held at: 18:30:0

--------------------------------
Process exited after 0.7356 seconds with return value 0
Press any key to continue . . .
```

```cpp
#include <iostream>
using namespace std;

class Basic {
private:
    static int static_num;
    int class_num;

public:
    Basic() {
        class_num = static_num++;
    }

    void Print() {
        cout << static_num << " " << class_num << endl;
    }
};

// Initialize static member
int Basic::static_num = 0;

int main() {
    Basic basic1;
    Basic basic2;
    Basic basic3;

    basic1.Print();
    basic2.Print();
    basic3.Print();

    return 0;
}
```

This program demonstrates the use of a static variable inside a class. The static_num variable is shared among all objects of the class, meaning it keeps increasing every time a new object is created. Each object gets its own class_num, which stores the value of static_num at the time the object is created.

Output:



After creating 3 objects, static_num becomes 3

Each object stored the value at creation:

basic1 → 0

basic2 → 1

basic3 → 2

But when printing, static_num is now shared and equals 3 for all prints.

Code

```cpp
#include <iostream>
using namespace std;

class Base {
private:
    int base_private;

public:
    Base() {
        base_private = 1;
    }

    void PrintBase() {
        cout << "Base private number: " << base_private << endl;
    }
};

class Derived : public Base {
private:
    int derived_private;

public:
    Derived() {
        derived_private = 4;
    }

    void PrintDerived() {
        cout << "Derived private number: " << derived_private << endl;
    }
};

int main() {
    Derived derived;

    derived.PrintBase();
    derived.PrintDerived();
```

This program demonstrates inheritance in C++ where a Derived class inherits from a Base class. The Base class contains a private integer initialized to 1 and a function to print it. The Derived class contains its own private integer initialized to 4 and a function to display it. When an object of the derived class is created, both constructors run and the program prints values from both the base and derived parts of the object using their respective functions.

output:

```
Base private number: 1
Derived private number: 4

--------------------------------
Process exited after 0.2838 seconds with return value 0
Press any key to continue . . .
```

The output shows two values because the object has both a Base part and a Derived part. First, the Base constructor sets base_private to 1, then the Derived constructor sets derived_private to 4. So when you call PrintBase(), it prints 1, and when you call PrintDerived(), it prints 4.

```cpp
#include <iostream>
using namespace std;

class Base {
private:
    int base_private;

protected:
    int base_protected;

public:
    int base_public;

    void setBaseNum(int n1, int n2, int n3) {
        base_private = n1;
        base_protected = n2;
        base_public = n3;
    }

    void printBase() {
        cout << base_private << " " << base_protected << " " << base_public << endl;
    }
};

class Derived : public Base {
public:
    void setBaseNum(int n1, int n2, int n3) {
        base_protected = n2;
        base_public = n3;
    }
};

int main() {
    Base base;
    base.setBaseNum(1, 2, 3);
    base.printBase();

    return 0;
```

This program shows access control in inheritance. The Base class has private, protected, and public variables. Private data can only be used inside the Base class, protected data can be used in the Base and Derived classes, and public data can be accessed anywhere. The Derived class can modify only the protected and public members of the Base class, not the private one. The program sets values using a function and prints them from the Base class.

OUTPUT

```
1 2 3

--------------------------------
Process exited after 0.2948 seconds with return value 0
Press any key to continue . . .
```

The program prints the three values stored in the Base object after they are set. The private value is 1, the protected value is 2, and the public value is 3.

Code

```cpp
#include <iostream>
using namespace std;

class Base {
private:
    int base_num;

public:
    Base() {
        base_num = 1;
        cout << "Constructor of base class" << endl;
    }

    Base(int n1) {
        base_num = n1;
        cout << "Constructor of base class" << endl;
    }
};

class Derived : public Base {
private:
    int derived_num;

public:
    Derived(int n) {
        derived_num = n;
        cout << "Constructor of derived class" << endl;
    }

    Derived(int n1, int n2) : Base(n1) {
        derived_num = n2;
        cout << "Constructor of derived class" << endl;
    }
};

int main() {
    Derived* ptr1 = new Derived(2);
    Derived* ptr2 = new Derived(1, 2);
```

This program demonstrates constructors in inheritance. The Base class has two constructors, one default and one parameterized. The Derived class also has two constructors. When a Derived object is created, the Base constructor is called first before the Derived constructor. In the second constructor, the Base constructor is explicitly called using : Base(n1).

OUTPUT

```
Constructor of base class
Constructor of derived class
Constructor of base class
Constructor of derived class

--------------------------------
Process exited after 0.2935 seconds with return value 0
Press any key to continue . . .
```

For Derived(2), the Base default constructor runs first, then the Derived constructor runs.

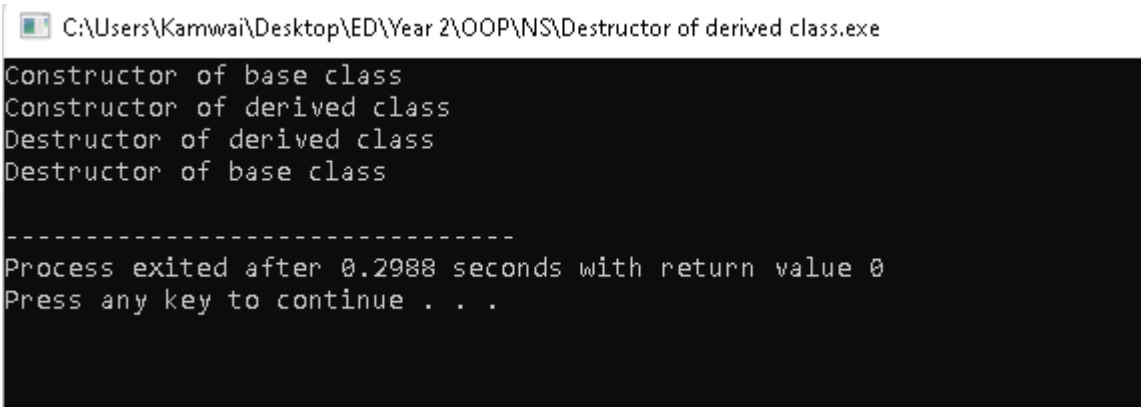
For Derived(1,2), the Base parameterized constructor runs first, then the Derived constructor runs.

CODE

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Constructor of base class" << endl;
    }

    ~Base() {
        cout << "Destructor of base class" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Constructor of derived class" << endl;
    }

    ~Derived() {
        cout << "Destructor of derived class" << endl;
    }
};

int main() {
    Derived *ptr1 = new Derived();

    delete ptr1;

    return 0;
}
```

This program shows constructors and destructors in inheritance. When a Derived object is created using new, first the Base constructor runs, then the Derived constructor. When the object is deleted using delete, the destructors run in reverse order: first Derived destructor, then Base destructor.

OUTPUT



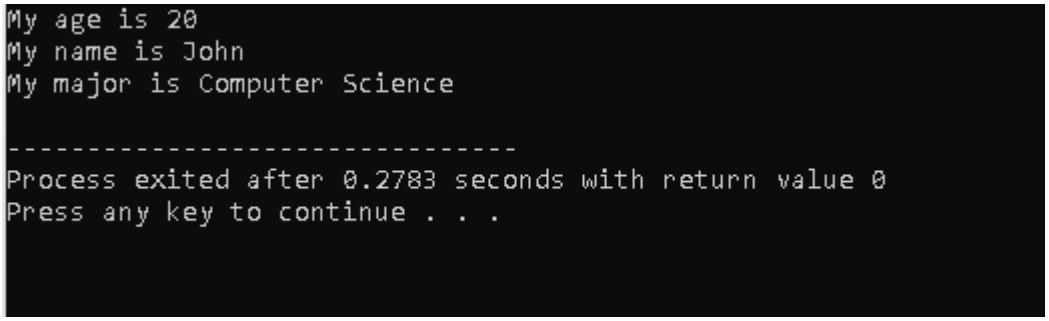
When the object is created, Base constructor runs first, then Derived constructor. When it is deleted, Derived destructor runs first, then Base destructor.

CODE

```
#include <iostream>
#include <cstring>
using namespace std;

class Person {
private:
    char *name;
    int age;

public:
    Person(int age, const char *name) {
        this->age = age;

        this->name = new char[strlen(name) + 1];
        strcpy(this->name, name);
    }

    ~Person() {
        delete[] name;
    }

    void PrintPerson() {
        cout << "My age is " << age << endl;
        cout << "My name is " << name << endl;
    }
};

class Student : public Person {
private:
    char *major;

public:
    Student(int age, const char *name, const char *major)
        : Person(age, name) {

        this->major = new char[strlen(major) + 1];
        strcpy(this->major, major);
    }
    }
};

int main() {
    Student s(20, "John", "Computer Science");

    s.PrintStudent();

    return 0;
}
```

This program demonstrates dynamic memory allocation and inheritance. The Person class stores a name and age using dynamically allocated memory, and the Student class inherits from Person while adding a major field, also stored dynamically. Constructors allocate memory, and destructors free it to prevent memory leaks. The student object prints all its information using both parent and child class functions.

OUTPUT



The program prints the person's age and name first (from the base class), then prints the student's major (from the derived class)

Multiple inheritance code

```cpp
#include <iostream>
using namespace std;

class Midl {
private:
    int number;

public:
    Midl(int num) {
        number = num;
    }

    void Midl_function() {
        cout << "Midl number is " << number << endl;
    }
};

class Mid2 {
private:
    int number;

public:
    Mid2(int num) {
        number = num;
    }

    void Mid2_function() {
        cout << "Mid2 number is " << number << endl;
    }
};

class Final : public Midl, public Mid2 {
public:
    Final(int num1, int num2)
        : Midl(num1), Mid2(num2) {}
};

int main() {
```

```cpp
int main() {
    Final f(1, 2);

    f.Midl_function();
    f.Mid2_function();

    return 0;
}
```

This program demonstrates multiple inheritance in C++. The class Final inherits from both Midl and Mid2. Each parent class stores its own integer value. The Final constructor passes values to both base classes using an initialization list. The program then calls functions from both base classes to display their stored values.

OUTPUT

```
Midl number is 1
Mid2 number is 2

--------------------------------
Process exited after 0.432 seconds with return value 0
Press any key to continue . . .
```

The object Final f(1, 2) stores 1 in Midl and 2 in Mid2. When the functions are called, each class prints its own value

Ambiguity of Multiple inheritance

```cpp
#include <iostream>
using namespace std;

class Midl {
private:
    int number;

public:
    Midl(int num) {
        number = num;
    }

    void Midl_function() {
        cout << "Midl number is " << number << endl;
    }
};

class Mid2 {
private:
    int number;

public:
    Mid2(int num) {
        number = num;
    }

    void Mid2_function() {
        cout << "Mid2 number is " << number << endl;
    }
};

class Final : public Midl, public Mid2 {
public:
    Final(int num1, int num2)
        : Midl(num1), Mid2(num2) {}
};

int main() {
    Final f(1, 2);

    f.Midl_function();
    f.Mid2_function();

    return 0;
}
```

This program demonstrates multiple inheritance in C++ where the class Final inherits from both Midl and Mid2. Each parent class stores its own integer value and has a function to print it. The Final class constructor passes values to both base classes using an initialization list.

OUTPUT

```
Midl number is 1
Mid2 number is 2

--------------------------------
Process exited after 0.3631 seconds with return value 0
Press any key to continue . . .
```

The object Final f(1, 2) creates two separate values, one for each parent class. When the functions are called, each class prints its own stored number.

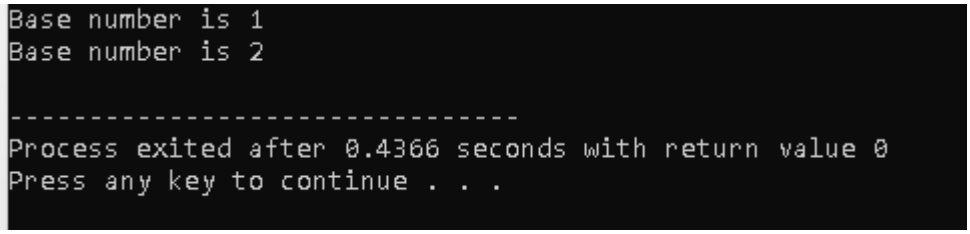
Code 2

```cpp
#include <iostream>
using namespace std;

class Base {
private:
    int number;

public:
    Base(int num) {
        number = num;
    }

    void Base_function() {
        cout << "Base number is " << number << endl;
    }
};

class Midl : public Base {
public:
    Midl(int num) : Base(num) {}
};

class Mid2 : public Base {
public:
    Mid2(int num) : Base(num) {}
};

class Final : public Midl, public Mid2 {
public:
    Final(int num1, int num2)
        : Midl(num1), Mid2(num2) {}
};

int main() {
    Final f(1, 2);

```

```
37      f.Midl::Base_function();
38      f.Mid2::Base_function();
39
40      return 0;
41  }
```

This program demonstrates multiple inheritance with a shared base class (diamond structure without virtual inheritance). The class Base stores a number and prints it. Midl and Mid2 both inherit from Base, and Final inherits from both of them. Because of this structure, Final contains two separate copies of Base, one from each parent class.

OUTPUT



The object Final f(1, 2) creates two separate Base objects: one through Midl and one through Mid2. When the functions are called, each path prints its own stored value.

VIRTUAL INHRITANCE CODE

```cpp
#include <iostream>
using namespace std;

class Base {
private:
    int number;

public:
    Base(int num) {
        number = num;
    }

    void Base_function() {
        cout << "Base number is " << number << endl;
    }
};

class Midl : virtual public Base {
public:
    Midl(int num) : Base(num) {}
};

class Mid2 : virtual public Base {
public:
    Mid2(int num) : Base(num) {}
};

class Final : public Midl, public Mid2 {
public:
    Final(int num) : Base(num), Midl(num), Mid2(num) {}
};

int main() {
    Final f(1);

    f.Base_function();
```

This program demonstrates virtual inheritance in C++. The Base class contains a number and a function to print it. Both Midl and Mid2 inherit from Base using virtual inheritance, which ensures that only one shared copy of Base exists inside Final. The Final class initializes the Base constructor directly to avoid duplication issues.

```
Base number is 1

--------------------------------
Process exited after 0.3681 seconds with return value 0
Press any key to continue . . .
```

Because virtual inheritance ensures only one Base exists inside Final, the value is set once and shared. So when Base_function() is called, it prints.